Functions and Pointers

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Functions, Pointers and Arrays

1. Which of the following is false?

```
int a[10] = {1, 2, 3} ; int *p = a;
int foo(int *q) { return (int)q; }
```

a) sizeof(a) == sizeof(p)

b) *a == *p

c) a == p

d) foo(a) == foo(p)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Functions, Pointers and Arrays

1. Which of the following is false?

```
int a[10] = {1, 2, 3} ; int *p = a;
int foo(int *q) { return (int)q; }
```

a) sizeof(a) == sizeof(p)

b) *a == *p

c) a == p

d) foo(a) == foo(p)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Functions, Pointers and Arrays

2. Comment :

```
void disp(int* f, int* l) {
    while(f != l) {
        printf("%d\n", *f); ++f;
    }
}
```

int a[] = {1, 2, 3, 4, 5}; disp(a, a + 5);

a) Syntax error; size of the array not specified while evaluating

b) Runtime error; a + 5 is a dangling pointer

c) Perfectly correct; no error

d) Runtime error; dangling pointer has been dereferenced

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Functions, Pointers and Arrays

2. Comment :

```
void disp(int* f, int* l) {
    while(f != l) {
        printf("%d\n", *f); ++f;
    }
}
```

int a[] = {1, 2, 3, 4, 5}; disp(a, a + 5);

a) Syntax error; size of the array not specified while evaluating

b) Runtime error; a + 5 is a dangling pointer

c) Perfectly correct; no error

d) Runtime error; dangling pointer has been dereferenced

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Functions, Pointers and Arrays

3. Comment :

```
void what(int* x, int* y) {

    *x = *x ^ *y;

    *y = *x ^ *y;

    *x = *x ^ *y;

}
```

int a = 10; int b = 20; what(&a, &a); What will be the values of a and b?

a) 10 20

b) 20 10

c) 0 0

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Functions, Pointers and Arrays

3. Comment :

```
void what(int* x, int* y) {

    *x = *x ^ *y;

    *y = *x ^ *y;

    *x = *x ^ *y;

}
```

int a = 10; int b = 20; what(&a, &a); What will be the values of a and b?

a) 10 20

b) 20 10

c) 0 0

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Functions, Pointers and Arrays

4. Comment :

```
void what() {
    int *p;
    {
        int a = 10; p = &a;
    }
    printf("%d \n", *p);
}
```

a) prints 10

b) Creates a garbage

c) Creates a dangling reference and dereferences it

d) Creates aliases

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Functions, Pointers and Arrays

4. Comment :

```
void what() {
    int *p;
    {
        int a = 10; p = &a;
    }
    printf("%d \n", *p);
}
```

a) prints 10

b) Creates a garbage

c) Creates a dangling reference and dereferences it

d) Creates aliases

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Functions, Pointers and Arrays

5. Comment :

```
void what() {

    int *p = (int*)malloc(sizeof(int)); // assume malloc succeeds

    *p = 111;

}
```

a) No error; memory reclaimed when the function terminates

b) Creates a garbage and therefore memory leak

c) Creates a dangling reference

d) Creates aliases

© Confidential Information of PES University. All Rights Reserved - 2026

Revision: Functions

- Associates a sequence of values called arguments with a value the result
- Arguments are copied to the corresponding parameters on a function call
- 'C' supports parameter passing by value
- To change an argument through a parameter, a pointer to the argument need to be passed
- When a function is called, an activation record or stack frame is created and all parameters, local variables, temporary variables, return address are stored on the stack
- When the function returns, the activation record is removed
- On a function call, the controlled is transferred to the beginning of the function
- On return, the instruction following the function call gets executed

© Confidential Information of PES University. All Rights Reserved - 2026

Revision: Functions

- A function calling itself or indirectly is called a recursion
- "if and recursion" is equivalent to an iterative loop
- A recursive function requires "escape hatch" or "base case" to stop recursive calls
- Recursive functions might take more time, but they are easy to develop if the underlying data structure is recursive.

© Confidential Information of PES University. All Rights Reserved - 2026

Revision: Functions

- Divide and conquer
- Sharing
- Maintenance
- Reliability
- Reuse

© Confidential Information of PES University. All Rights Reserved - 2026

Revision: Pointers

- Holds an address of another entity
- is not same as integer
- Has a type ; pointer to int is not same as pointer to double
- Pointer supports 3 arithmetic operation
    - Add an int to a pointer => results in a pointer
    - Subtract an int from a pointer => results in a pointer
    - Take difference between two pointers => results in an int
- Parameter of a function has to be a pointer if the correnspoding argument should be changed by dereferencing

© Confidential Information of PES University. All Rights Reserved - 2026

Revision : Pointers

- Has three points to note
- Aliasing : two pointers point to the same location
    - changing the referent through one also affects referencing through the other
- Dangling pointer : no location; but have access to it
    - Dangling pointers are not dangerous
    - dereferencing a dangling pointer is an undefined behaviour
    - program crashes if you are lucky!
- Garbage : location; but no access to it
    - location is lost until the program terminates
    - so this is called memory leak
    - 'C' does not support garbage collection

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)